AISC Practical: All 23 Experiments

Artificial Intelligence and Soft Computing, Python code (all programs tested)

Quick cheat sheet

Topic	Core formula / idea
Perceptron (Exp 1-4)	w = w + c*(t-o)*x (continuous: multiply by f'(net))
Hebbian (Exp 5-8)	w = w + c*o*x (unsupervised, no target)
Unipolar activation	discrete: 1 if net>0 else 0 | continuous: 1/(1+e^-net), f' = o(1-o)
Bipolar activation	discrete: 1 if net>0 else -1 | continuous: 2/(1+e^-net)-1, f' = 0.5(1-o^2)
Search (Exp 9-12)	DLS: depth cutoff | DFID: DLS with growing limit | Hill: best neighbour | A*: f=g+h
Games (Exp 13-14)	Minimax alternates max/min; alpha-beta prunes when alpha >= beta
Defuzzification (17-21)	Weighted avg, Centroid, Centre of Sum, Height, Centre of Area (bisector)
Competitive (22-23)	WTA: update winner only | LVQ: attract if same class, repel if different

Note: outputs shown for Exp 3, 4 and 11 use random starting values, so your numbers will differ slightly.

Experiment 1: Discrete perceptron, unipolar binary activation

w = w + c*(t-o)*x, only when o != t. Stop when a full pass has 0 errors.

CODE

```
import numpy as np

X = np.array([[0,0,1],[0,1,1],[1,0,1],[1,1,1]])   # last column = bias
d = np.array([0,0,0,1])                            # AND targets
w = np.zeros(3)
c = 0.1

f = lambda net: 1 if net > 0 else 0                # unipolar binary

for epoch in range(100):
    errors = 0
    for x, t in zip(X, d):
        o = f(w @ x)
        if o != t:
            w = w + c * (t - o) * x
            errors += 1
    if errors == 0:
        break

print("Epochs:", epoch + 1, "Weights:", w)
```

SAMPLE OUTPUT

```
Epochs: 6 Weights: [ 0.2  0.1 -0.2]
```

Experiment 2: Discrete perceptron, bipolar binary activation

Same as Exp 1: inputs/targets use -1 and +1, f = 1 if net>0 else -1.

CODE

```
import numpy as np

X = np.array([[-1,-1,1],[-1,1,1],[1,-1,1],[1,1,1]])
d = np.array([-1,-1,-1,1])
w = np.zeros(3)
c = 0.1

f = lambda net: 1 if net > 0 else -1               # bipolar binary

for epoch in range(100):
    errors = 0
    for x, t in zip(X, d):
        o = f(w @ x)
        if o != t:
            w = w + c * (t - o) * x
            errors += 1
    if errors == 0:
        break

print("Epochs:", epoch + 1, "Weights:", w)
```

SAMPLE OUTPUT

```
Epochs: 3 Weights: [ 0.2  0.2 -0.2]
```

Experiment 3: Continuous perceptron, unipolar (sigmoid)

Update every example with derivative o(1-o). Stop when total error < threshold.

CODE

```
import numpy as np

X = np.array([[0,0,1],[0,1,1],[1,0,1],[1,1,1]])
d = np.array([0,0,0,1])
w = np.random.rand(3)
c = 1.0

f = lambda net: 1 / (1 + np.exp(-net))             # unipolar continuous

for epoch in range(20000):
    error = 0
    for x, t in zip(X, d):
```

```
        o = f(w @ x)
        w = w + c * (t - o) * o * (1 - o) * x      # f'(net) = o(1-o)
        error += 0.5 * (t - o) ** 2
    if error < 0.01:
        break

print("Epochs:", epoch + 1, "Weights:", w)
```

SAMPLE OUTPUT

```
Epochs: 553 Weights: [ 4.81753739  4.81082036 -7.30518317]
```

Experiment 4: Continuous perceptron, bipolar (sigmoid)

Activation 2/(1+e^-net)-1, derivative 0.5*(1-o^2).

CODE

```
import numpy as np

X = np.array([[-1,-1,1],[-1,1,1],[1,-1,1],[1,1,1]])
d = np.array([-1,-1,-1,1])
w = np.random.rand(3)
c = 0.5

f = lambda net: 2 / (1 + np.exp(-net)) - 1         # bipolar continuous

for epoch in range(20000):
    error = 0
    for x, t in zip(X, d):
        o = f(w @ x)
        w = w + c * (t - o) * 0.5 * (1 - o**2) * x # f'(net) = 0.5(1-o^2)
        error += 0.5 * (t - o) ** 2
    if error < 0.01:
        break

print("Epochs:", epoch + 1, "Weights:", w)
```

SAMPLE OUTPUT

```
Epochs: 171 Weights: [ 3.16593614  3.16805226 -3.16382423]
```

Experiment 5: Hebbian learning, unipolar binary

w = w + c*o*x. No target. Initial weights must be non-zero.

CODE

```
import numpy as np

X = np.array([[1, -2, 1.5, 0],
              [1, -0.5, -2, -1.5],
              [0, 1, -1, 1.5]])
w = np.array([1, -1, 0, 0.5], dtype=float)   # must be non-zero
c = 1

f = lambda net: 1 if net > 0 else 0   # unipolar binary

for i, x in enumerate(X, 1):
    net = w @ x
    o = f(net)
    w = w + c * o * x                          # Hebb: dw = c * o * x
    print(f"Step {i}: net={net:.3f}, o={o:.3f}, w={w}")
```

SAMPLE OUTPUT

```
Step 1: net=3.000, o=1.000, w=[ 2.  -3.   1.5  0.5]
Step 2: net=-0.250, o=0.000, w=[ 2.  -3.   1.5  0.5]
Step 3: net=-3.750, o=0.000, w=[ 2.  -3.   1.5  0.5]
```

Experiment 6: Hebbian learning, bipolar binary

Same Hebb rule, f = 1 if net>0 else -1.

CODE

```
import numpy as np

X = np.array([[1, -2, 1.5, 0],
              [1, -0.5, -2, -1.5],
              [0, 1, -1, 1.5]])
```

```
w = np.array([1, -1, 0, 0.5], dtype=float)   # must be non-zero
c = 1

f = lambda net: 1 if net > 0 else -1   # bipolar binary

for i, x in enumerate(X, 1):
    net = w @ x
    o = f(net)
    w = w + c * o * x                          # Hebb: dw = c * o * x
    print(f"Step {i}: net={net:.3f}, o={o:.3f}, w={w}")
```

SAMPLE OUTPUT

```
Step 1: net=3.000, o=1.000, w=[ 2.  -3.   1.5  0.5]
Step 2: net=-0.250, o=-1.000, w=[ 1.  -2.5  3.5  2. ]
Step 3: net=-3.000, o=-1.000, w=[ 1.  -3.5  4.5  0.5]
```

Experiment 7: Hebbian learning, unipolar continuous

Same Hebb rule, f = sigmoid.

CODE

```
import numpy as np

X = np.array([[1, -2, 1.5, 0],
              [1, -0.5, -2, -1.5],
              [0, 1, -1, 1.5]])
w = np.array([1, -1, 0, 0.5], dtype=float)   # must be non-zero
c = 1

f = lambda net: 1 / (1 + np.exp(-net))   # unipolar continuous

for i, x in enumerate(X, 1):
    net = w @ x
    o = f(net)
    w = w + c * o * x                          # Hebb: dw = c * o * x
    print(f"Step {i}: net={net:.3f}, o={o:.3f}, w={w}")
```

SAMPLE OUTPUT

```
Step 1: net=3.000, o=0.953, w=[ 1.95257413 -2.90514825  1.42886119  0.5       ]
Step 2: net=-0.203, o=0.450, w=[ 2.40210307 -3.12991273  0.5298033  -0.17429342]
Step 3: net=-3.921, o=0.019, w=[ 2.40210307 -3.11047969  0.51037026 -0.14514386]
```

Experiment 8: Hebbian learning, bipolar continuous

Same Hebb rule, f = 2*sigmoid - 1.

CODE

```
import numpy as np

X = np.array([[1, -2, 1.5, 0],
              [1, -0.5, -2, -1.5],
              [0, 1, -1, 1.5]])
w = np.array([1, -1, 0, 0.5], dtype=float)   # must be non-zero
c = 1

f = lambda net: 2 / (1 + np.exp(-net)) - 1   # bipolar continuous

for i, x in enumerate(X, 1):
    net = w @ x
    o = f(net)
    w = w + c * o * x                          # Hebb: dw = c * o * x
    print(f"Step {i}: net={net:.3f}, o={o:.3f}, w={w}")
```

SAMPLE OUTPUT

```
Step 1: net=3.000, o=0.905, w=[ 1.90514825 -2.81029651  1.35772238  0.5       ]
Step 2: net=-0.155, o=-0.077, w=[ 1.82772936 -2.77158706  1.51256017  0.61612834]
Step 3: net=-3.360, o=-0.933, w=[ 1.82772936 -3.70444568  2.44541878 -0.78315958]
```

Experiment 9: Depth Limited Search

DFS that stops going deeper when limit reaches 0.

CODE

```
graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'],
         'D': [], 'E': ['H'], 'F': [], 'G': [], 'H': []}
```

```
def dls(node, goal, limit, path=()):
    path = path + (node,)
    if node == goal:
        return list(path)
    if limit == 0:                      # depth limit reached: cut off
        return None
    for child in graph[node]:
        result = dls(child, goal, limit - 1, path)
        if result:
            return result
    return None

goal = 'H'
for limit in (1, 2, 3):
    print("Limit", limit, "->", dls('A', goal, limit))
```

SAMPLE OUTPUT

```
Limit 1 -> None
Limit 2 -> None
Limit 3 -> ['A', 'B', 'E', 'H']
```

Experiment 10: DFID (Depth First Iterative Deepening)

Run depth-limited search with limit 0, 1, 2, ... until goal is found.

CODE

```
graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'],
         'D': [], 'E': ['H'], 'F': [], 'G': [], 'H': []}

def dls(node, goal, limit, path=()):
    path = path + (node,)
    if node == goal:
        return list(path)
    if limit == 0:
        return None
    for child in graph[node]:
        result = dls(child, goal, limit - 1, path)
        if result:
            return result
    return None

def dfid(start, goal, max_depth=10):
    for depth in range(max_depth + 1):       # 0, 1, 2, ...
        print("Trying depth", depth)
        result = dls(start, goal, depth)
        if result:
            return result, depth
    return None, None

path, depth = dfid('A', 'H')
print("Path:", path, "found at depth", depth)
```

SAMPLE OUTPUT

```
Trying depth 0
Trying depth 1
Trying depth 2
Trying depth 3
Path: ['A', 'B', 'E', 'H'] found at depth 3
```

Experiment 11: Hill Climbing

Move to the best neighbour; stop when no neighbour is better (peak).

CODE

```
import random

def f(x):                                   # objective function (maximise)
    return -(x - 5) ** 2 + 25

x = random.randint(-10, 20)                 # random start
print("Start:", x, "f =", f(x))

while True:
    neighbours = [x - 1, x + 1]
    best = max(neighbours, key=f)           # best neighbour
    if f(best) <= f(x):                     # no better neighbour: peak reached
```

```
            break
        x = best
        print("Move to", x, "f =", f(x))

    print("Peak at x =", x, "f =", f(x))
```

SAMPLE OUTPUT

```
Start: 5 f = 25
Peak at x = 5 f = 25
```

Experiment 12: A* algorithm

f(n) = g(n) + h(n). Priority queue ordered by f.

CODE

```
import heapq

graph = {'S': {'A': 1, 'B': 4}, 'A': {'B': 2, 'C': 5, 'D': 12},
         'B': {'C': 2}, 'C': {'D': 3}, 'D': {}}
h = {'S': 7, 'A': 6, 'B': 4, 'C': 2, 'D': 0}      # heuristic estimates

def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]   # (f, g, node, path)
    best_g = {}
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        if node == goal:
            return path, g
        if node in best_g and best_g[node] <= g:
            continue
        best_g[node] = g
        for nb, cost in graph[node].items():
            g2 = g + cost
            heapq.heappush(open_list, (g2 + h[nb], g2, nb, path + [nb]))
    return None, float('inf')

path, cost = a_star('S', 'D')
print("Path:", path, "Cost:", cost)
```

SAMPLE OUTPUT

```
Path: ['S', 'A', 'B', 'C', 'D'] Cost: 8
```

Experiment 13: Min-Max algorithm

MAX levels take max of children, MIN levels take min.

CODE

```
# leaves are utilities; levels alternate MAX, MIN, MAX ...
tree = [[[3, 5], [6, 9]], [[1, 2], [0, -1]]]

def minimax(node, is_max):
    if not isinstance(node, list):            # leaf
        return node
    values = [minimax(child, not is_max) for child in node]
    return max(values) if is_max else min(values)

print("Minimax value at root:", minimax(tree, True))
```

SAMPLE OUTPUT

```
Minimax value at root: 5
```

Experiment 14: Alpha-Beta pruning

Minimax plus alpha/beta bounds; prune when alpha >= beta.

CODE

```
tree = [[[3, 5], [6, 9]], [[1, 2], [0, -1]]]
pruned = 0

def alphabeta(node, is_max, alpha, beta):
    global pruned
    if not isinstance(node, list):
        return node
    if is_max:
        value = float('-inf')
```

```
        for child in node:
            value = max(value, alphabeta(child, False, alpha, beta))
            alpha = max(alpha, value)
            if alpha >= beta:                # prune remaining children
                pruned += 1
                break
        return value
    else:
        value = float('inf')
        for child in node:
            value = min(value, alphabeta(child, True, alpha, beta))
            beta = min(beta, value)
            if alpha >= beta:
                pruned += 1
                break
        return value

print("Value:", alphabeta(tree, True, float('-inf'), float('inf')))
print("Number of prunings:", pruned)
```

SAMPLE OUTPUT

```
Value: 5
Number of prunings: 2
```

Experiment 15: Fuzzy operations on two fuzzy sets

Union=max, Intersection=min, Complement=1-a, Alg. sum=a+b-ab, Alg. product=ab.

CODE

```
A = {'x1': 0.2, 'x2': 0.5, 'x3': 0.7, 'x4': 1.0}
B = {'x1': 0.6, 'x2': 0.3, 'x3': 0.9, 'x4': 0.4}

union        = {x: max(A[x], B[x])        for x in A}
intersection = {x: min(A[x], B[x])        for x in A}
complement_A = {x: round(1 - A[x], 2)     for x in A}
alg_sum      = {x: round(A[x] + B[x] - A[x]*B[x], 2) for x in A}
alg_product  = {x: round(A[x] * B[x], 2)  for x in A}
difference   = {x: round(min(A[x], 1 - B[x]), 2) for x in A}   # A - B
bounded_sum  = {x: round(min(1, A[x] + B[x]), 2) for x in A}

print("Union          :", union)
print("Intersection   :", intersection)
print("Complement of A:", complement_A)
print("Algebraic sum  :", alg_sum)
print("Algebraic prod :", alg_product)
print("Difference A-B :", difference)
print("Bounded sum    :", bounded_sum)
```

SAMPLE OUTPUT

```
Union          : {'x1': 0.6, 'x2': 0.5, 'x3': 0.9, 'x4': 1.0}
Intersection   : {'x1': 0.2, 'x2': 0.3, 'x3': 0.7, 'x4': 0.4}
Complement of A: {'x1': 0.8, 'x2': 0.5, 'x3': 0.3, 'x4': 0.0}
Algebraic sum  : {'x1': 0.68, 'x2': 0.65, 'x3': 0.97, 'x4': 1.0}
Algebraic prod : {'x1': 0.12, 'x2': 0.15, 'x3': 0.63, 'x4': 0.4}
Difference A-B : {'x1': 0.2, 'x2': 0.5, 'x3': 0.1, 'x4': 0.6}
Bounded sum    : {'x1': 0.8, 'x2': 0.8, 'x3': 1, 'x4': 1}
```

Experiment 16: Composition of fuzzy relations

Max-min: max_k min(R,S). Max-product: max_k R*S.

CODE

```
import numpy as np

R = np.array([[0.7, 0.5],
              [0.8, 0.4]])                # X x Y
S = np.array([[0.9, 0.6, 0.2],
              [0.1, 0.7, 0.5]])           # Y x Z

def max_min(R, S):
    m, n, p = R.shape[0], R.shape[1], S.shape[1]
    T = np.zeros((m, p))
    for i in range(m):
        for j in range(p):
            T[i, j] = max(min(R[i, k], S[k, j]) for k in range(n))
    return T
```

```
def max_product(R, S):
    m, n, p = R.shape[0], R.shape[1], S.shape[1]
    T = np.zeros((m, p))
    for i in range(m):
        for j in range(p):
            T[i, j] = max(R[i, k] * S[k, j] for k in range(n))
    return T

print("Max-Min composition:\n", max_min(R, S))
print("Max-Product composition:\n", max_product(R, S))
```

SAMPLE OUTPUT

```
Max-Min composition:
 [[0.7 0.6 0.5]
 [0.8 0.6 0.4]]
Max-Product composition:
 [[0.63 0.42 0.25]
 [0.72 0.48 0.2 ]]
```

Experiment 17: Weighted average defuzzification

x* = sum(mu_i * x_i) / sum(mu_i), using peak of each set.

CODE

```
# Weighted average: x* = sum(mu_i * x_i) / sum(mu_i)
# x_i = peak (mean) value of each output fuzzy set, mu_i = its firing strength
x  = [2.0, 5.0, 8.0]          # peaks of Low, Medium, High
mu = [0.3, 0.7, 0.4]          # membership (clipped) heights

x_star = sum(m * xi for m, xi in zip(mu, x)) / sum(mu)
print("Weighted average defuzzified value:", round(x_star, 3))
```

SAMPLE OUTPUT

```
Weighted average defuzzified value: 5.214
```

Experiment 18: Centroid defuzzification

Aggregate with MAX, then x* = sum(x*mu) / sum(mu).

CODE

```
import numpy as np

def tri(x, a, b, c):                      # triangular membership function
    return np.maximum(np.minimum((x - a) / (b - a), (c - x) / (c - b)), 0)

x = np.linspace(0, 10, 1001)              # universe of discourse
strength = {"Low": 0.3, "Med": 0.7, "High": 0.4}      # firing strengths
sets = {"Low": tri(x, -1, 2, 5), "Med": tri(x, 2, 5, 8), "High": tri(x, 5, 8, 11)}
clipped = {k: np.minimum(sets[k], strength[k]) for k in sets}   # clip at strength

# Centroid: aggregate with MAX, then x* = sum(x*mu) / sum(mu)
agg = np.maximum.reduce(list(clipped.values()))
x_star = np.sum(x * agg) / np.sum(agg)
print("Centroid:", round(x_star, 3))
```

SAMPLE OUTPUT

```
Centroid: 5.228
```

Experiment 19: Centre of Sum defuzzification

Aggregate with SUM (overlaps counted twice), then centroid.

CODE

```
import numpy as np

def tri(x, a, b, c):                      # triangular membership function
    return np.maximum(np.minimum((x - a) / (b - a), (c - x) / (c - b)), 0)

x = np.linspace(0, 10, 1001)              # universe of discourse
strength = {"Low": 0.3, "Med": 0.7, "High": 0.4}      # firing strengths
sets = {"Low": tri(x, -1, 2, 5), "Med": tri(x, 2, 5, 8), "High": tri(x, 5, 8, 11)}
clipped = {k: np.minimum(sets[k], strength[k]) for k in sets}   # clip at strength

# Centre of Sum: aggregate with SUM (overlaps counted twice)
total = sum(clipped.values())
```

```
x_star = np.sum(x * total) / np.sum(total)
print("Centre of Sum:", round(x_star, 3))
```

SAMPLE OUTPUT

```
Centre of Sum: 5.199
```

Experiment 20: Height method defuzzification

x* = sum(peak_k * height_k) / sum(height_k).

CODE

```
import numpy as np

def tri(x, a, b, c):                        # triangular membership function
    return np.maximum(np.minimum((x - a) / (b - a), (c - x) / (c - b)), 0)

x = np.linspace(0, 10, 1001)                # universe of discourse
strength = {"Low": 0.3, "Med": 0.7, "High": 0.4}      # firing strengths
sets = {"Low": tri(x, -1, 2, 5), "Med": tri(x, 2, 5, 8), "High": tri(x, 5, 8, 11)}
clipped = {k: np.minimum(sets[k], strength[k]) for k in sets}   # clip at strength

# Height method: x* = sum(peak_k * height_k) / sum(height_k)
peak = {"Low": 2, "Med": 5, "High": 8}    # x where each set is maximum
num = sum(peak[k] * strength[k] for k in peak)
den = sum(strength[k] for k in peak)
print("Height method:", round(num / den, 3))
```

SAMPLE OUTPUT

```
Height method: 5.214
```

Experiment 21: Centre of Area (bisector) defuzzification

x* is the point that splits the aggregated area into two equal halves.

CODE

```
import numpy as np

def tri(x, a, b, c):                        # triangular membership function
    return np.maximum(np.minimum((x - a) / (b - a), (c - x) / (c - b)), 0)

x = np.linspace(0, 10, 1001)                # universe of discourse
strength = {"Low": 0.3, "Med": 0.7, "High": 0.4}      # firing strengths
sets = {"Low": tri(x, -1, 2, 5), "Med": tri(x, 2, 5, 8), "High": tri(x, 5, 8, 11)}
clipped = {k: np.minimum(sets[k], strength[k]) for k in sets}   # clip at strength

# Centre of Area (bisector): x that splits the area into two equal halves
agg = np.maximum.reduce(list(clipped.values()))
cum = np.cumsum(agg)
idx = np.argmax(cum >= cum[-1] / 2)       # first point with half the area
print("Centre of Area:", round(x[idx], 3))
```

SAMPLE OUTPUT

```
Centre of Area: 5.21
```

Experiment 22: Winner-Take-All learning

Only the neuron with the largest net is updated: w_j += alpha*(x - w_j).

CODE

```
import numpy as np

X = np.array([[1.0, 0.0], [0.9, 0.1], [0.0, 1.0], [0.1, 0.9]])
W = np.array([[0.6, 0.8], [0.8, 0.6]])              # one row per neuron
W = W / np.linalg.norm(W, axis=1, keepdims=True)    # normalise weights
alpha = 0.5

for epoch in range(20):
    for x in X:
        net = W @ x
        j = np.argmax(net)                          # winner takes all
        W[j] = W[j] + alpha * (x - W[j])            # only winner is updated
        W[j] = W[j] / np.linalg.norm(W[j])
    alpha *= 0.9                                    # decay learning rate

print("Final weights:\n", W)
```

```
for x in X:
    print(x, "-> cluster", np.argmax(W @ x))
```

SAMPLE OUTPUT

```
Final weights:
 [[0.05603716 0.99842868]
 [0.99842868 0.05603716]]
[1. 0.] -> cluster 1
[0.9 0.1] -> cluster 1
[0. 1.] -> cluster 0
[0.1 0.9] -> cluster 0
```

Experiment 23: Learning Vector Quantization (LVQ1)

Nearest prototype: same class -> move closer, different class -> move away.

CODE

```
import numpy as np

X = np.array([[0.1, 0.2], [0.2, 0.1], [0.8, 0.9], [0.9, 0.8]])
y = np.array([0, 0, 1, 1])                  # class labels of the training data
W = np.array([[0.4, 0.3], [0.6, 0.7]])      # prototype (codebook) vectors
wl = np.array([0, 1])                       # class label of each prototype
alpha = 0.3

for epoch in range(30):
    for x, t in zip(X, y):
        j = np.argmin(np.linalg.norm(W - x, axis=1))   # nearest prototype
        if wl[j] == t:
            W[j] = W[j] + alpha * (x - W[j])           # same class: pull closer
        else:
            W[j] = W[j] - alpha * (x - W[j])           # different class: push away
    alpha *= 0.9

print("Final prototypes:\n", W)
for x in X:
    j = np.argmin(np.linalg.norm(W - x, axis=1))
    print(x, "-> class", wl[j])
```

SAMPLE OUTPUT

```
Final prototypes:
 [[0.15213883 0.14859589]
 [0.85122043 0.84804485]]
[0.1 0.2] -> class 0
[0.2 0.1] -> class 0
[0.8 0.9] -> class 1
[0.9 0.8] -> class 1
```